

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 18:22 UTC

Overall Assessment

PR summary generation was skipped because Groq rate limited the request.

Merge readiness: unknown

Code health: 0/100

Findings Summary

Total findings: 12

Risk score: 46

Severity breakdown: Low 0, Medium 0, High 2, Critical 10

Type	Severity	Line	Message	Recommendation
Hardcoded Password	High	1	password = "admin123"	Store passwords in environment variables.
Hardcoded API Key	High	3	api_key = "123456789"	Store secrets securely.
SQL Injection	Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input())	Avoid dynamic code execution.
Dangerous Dynamic Execution	Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
Command Injection	Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
Hardcoded Credentials	Critical	1	Hardcoded password found in the code	Store sensitive information securely using environment variables or a secrets manager
Hardcoded Credentials	Critical	3	Hardcoded API key found in the code	Store sensitive information securely using environment variables or a secrets manager
SQL Injection	Critical	5	User input is directly concatenated into a SQL query, making it vulnerable to SQL injection attacks	Use parameterized queries or prepared statements to prevent SQL injection
Arbitrary Code Execution	Critical	7	The eval function is used with user input, allowing arbitrary code execution	Avoid using eval with user input and instead use a safer alternative, such as a parsing library

Type	Severity	Line	Message	Recommendation
Arbitrary Code Execution	Critical	9	The exec function is used, allowing arbitrary code execution	Avoid using exec and instead use a safer alternative, such as a parsing library or a templating engine
Command Injection	Critical	12	The subprocess.run function is used with the shell=True argument, making it vulnerable to command injection attacks	Use the subprocess.run function with the shell=False argument and pass the command and arguments as a list to prevent command injection

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
---------	----------	--------	------------------

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```